Mastering AKS RBAC and Azure AD Integration Essentials

Securing your Kubernetes cluster does not have to be complicated or scary. By combining Azure Active Directory with Kubernetes Role-Based Access Control, you can control exactly who touches your cluster using familiar company logins.

What you'll learn:

1. 1. Enable Azure AD RBAC on your AKS Cluster
2. 2. Assign Basic Cluster User Access
3. 3. Limit User Access to a Single Namespace
4. 4. Test Permissions Before Execution

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Enable Azure AD RBAC on your AKS Cluster

Integrating Azure Active Directory (Entra ID) delegates your cluster authentication directly to Azure. Instead of manually creating certificates or user accounts inside Kubernetes, developers log in using their standard company credentials. Enabling native Azure RBAC allows you to control Kubernetes permissions straight from Azure CLI or the portal.

```
$az aks create --resource-group myResourceGroup --name myCluster --enable-aad --enable-azure-rbac
```

AKS RBAC & Azure AD Integration: Secure Who Can Touch Your Cluster

Tuesday, November 10, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Assign Basic Cluster User Access

Before any engineer can run kubectl commands against an Azure AD integrated cluster, they need the Cluster User role. This role allows users to authenticate and get context credentials, but does not grant them permission to modify cluster resources until specific RBAC roles are assigned.

```
$az role assignment create --assignee "dev@company.com" --role "Azure Kubernetes Service Cluster User
```

AKS RBAC & Azure AD Integration: Secure Who Can Touch Your Cluster

Tuesday, November 10, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Limit User Access to a Single Namespace

To prevent accidental changes to critical systems, grant permissions inside a specific namespace rather than cluster-wide. By binding built-in Kubernetes ClusterRoles like 'view' or 'edit' to a user account within one namespace, you keep team members safely contained in their own project sandbox.

```
$kubectl create rolebinding user-dev-view --clusterrole=view --user="dev@company.com" --namespace=dev
```

AKS RBAC & Azure AD Integration: Secure Who Can Touch Your Cluster

Tuesday, November 10, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Test Permissions Before Execution

Unsure if your automation account or team member has access to perform a task? Kubernetes provides a built-in checking tool that evaluates your permissions without running the actual command. Using 'can-i' prevents failed deployment scripts and unintended access errors.

```
$kubectl auth can-i delete deployments --namespace development
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Assign roles to Azure AD groups rather than individual user accounts.
+ Use namespace-scoped RoleBindings instead of cluster-wide ClusterRoleBindings whenever possible.
+ Always enforce least privilege: start with 'view' access and upgrade only when necessary.
+ Regularly review cluster role assignments using 'az role assignment list'.
+ Use 'kubectl auth can-i' in CI/CD scripts to verify permission before running tasks.

Watch Out For

! Granting 'Cluster Admin' access to fix access errors quickly instead of diagnosing RBAC policies.
! Sharing raw administrator kubeconfig files over email or Slack.
! Forgetting to specify the '--namespace' flag when creating user RoleBindings.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh